PROJECT

Wahbo Sharif

ABSTRACT

- This project develops a dynamic model of a self-balancing car, performs a parameterization study on system stability, and designs a PID controller to stabilize the system.
- The plant model is implemented and tested in Simulink. Parameter variations, particularly inertia and damping are shown to affect transient response significantly.
- The PID controller improves stability, reduces overshoot, and ensures the car maintains an upright position.

INTRODUCTION

<u>The Challenge of Balance</u>

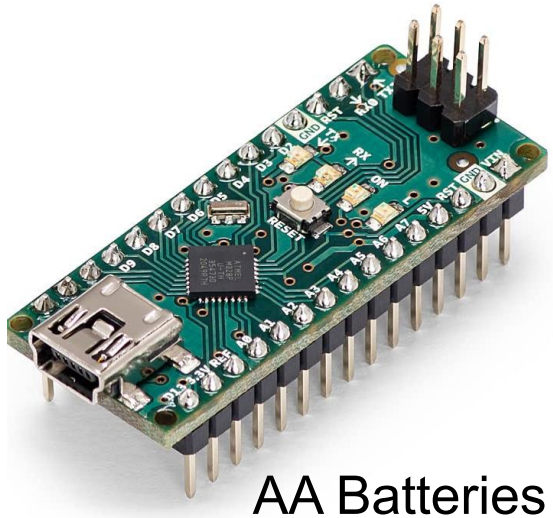
A self-balancing car behaves like an inverted pendulum, a classic unstable linear system. Without active control, small disturbances cause the system to diverge from equilibrium.

- Dynamics: A highly non-linear system linearized for control
- Control Strategy: PID control is essential to correct deviations in real-time
- Study Goal: Understand how kp, ki, and kd influence stability in the real world.

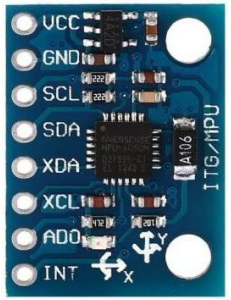
COMPONENTS

Lego Wheels

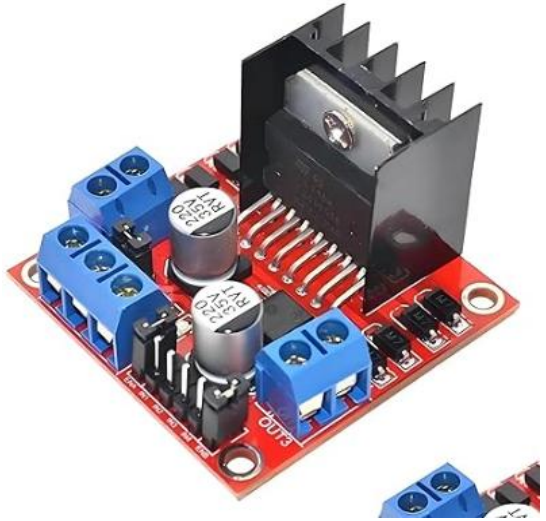
Arduino Nano



MPU-6050

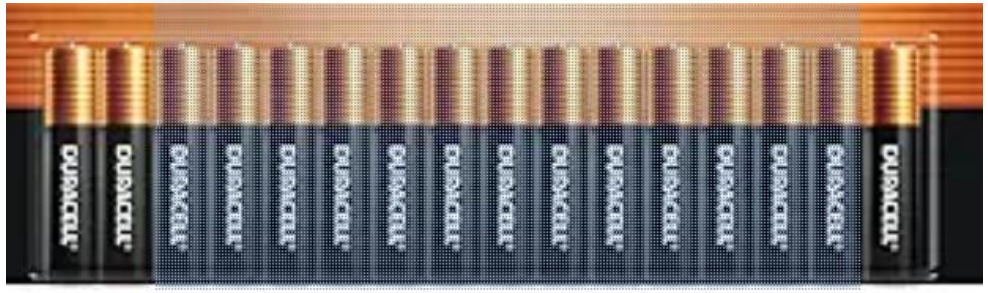


Motor driver L298N

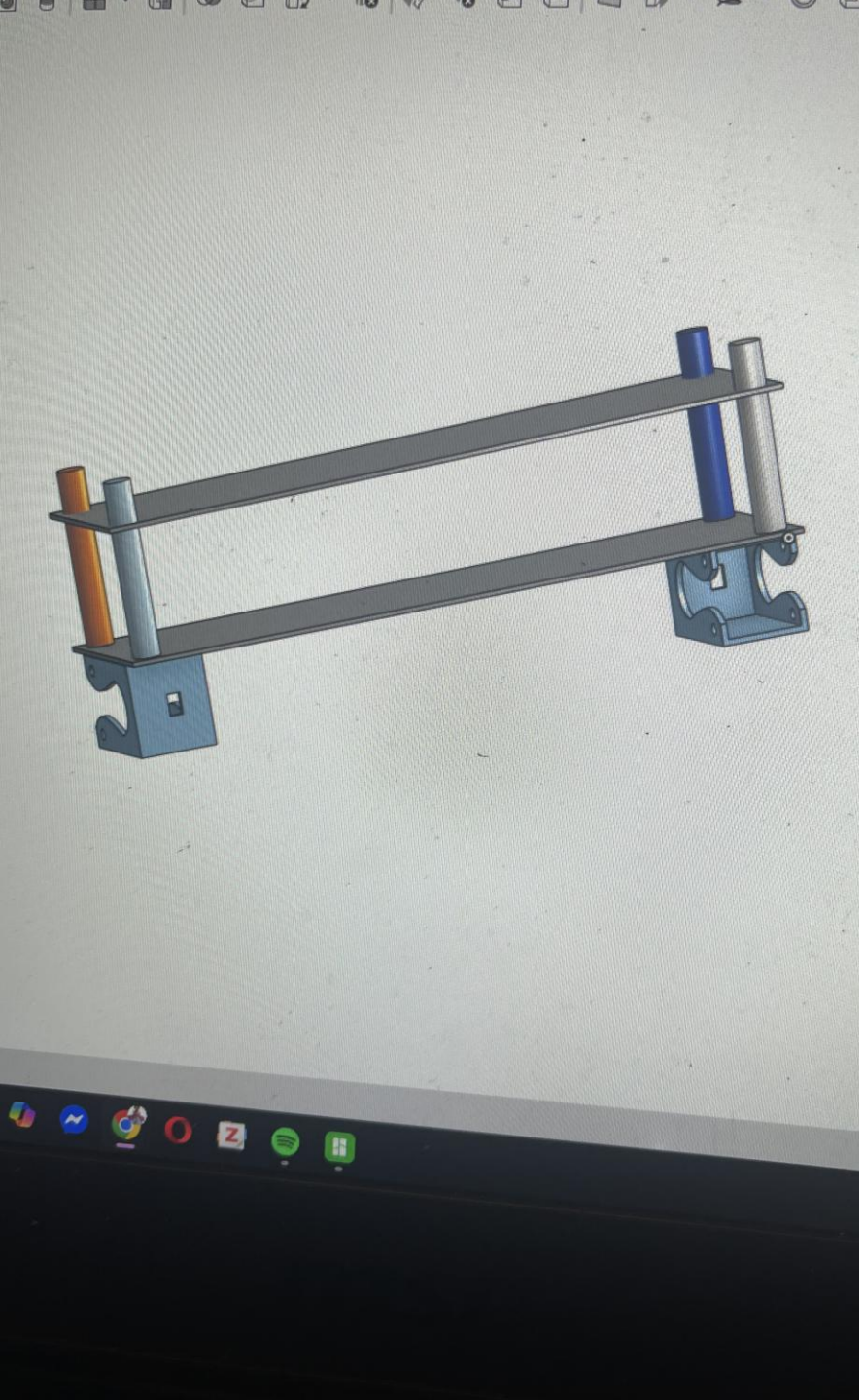


Jumper wires

AA Batteries

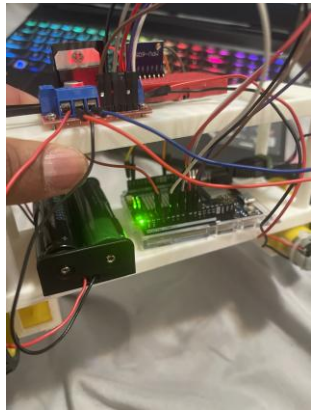


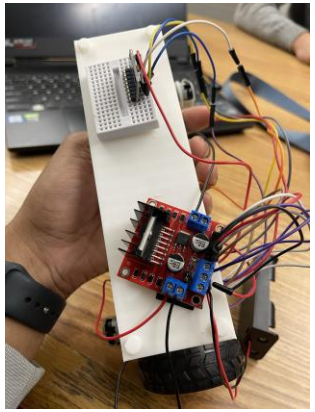
M2 bolts

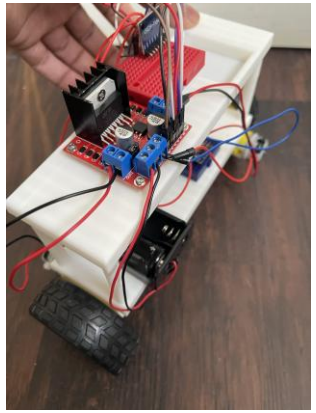


MODELING OF THE PLANT SYSTEM

- The plant is modeled as an inverted pendulum on a moving base.
- Design was created using Onshape.



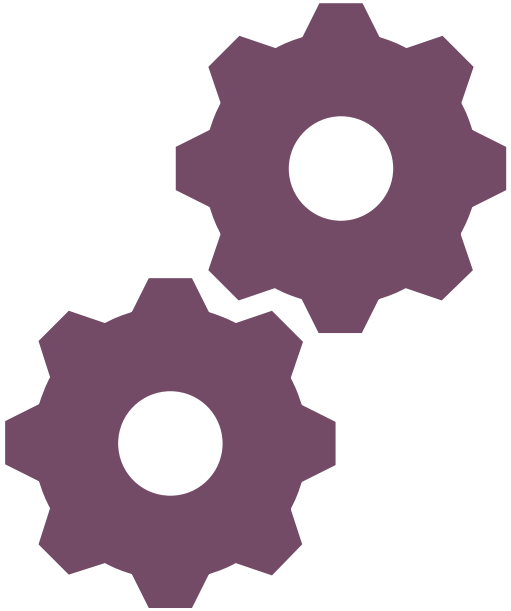




PARAMETERIZATION STUDY

- Changing the kd value makes the motor wobbly as kp and ki are 0.
- Changing the ki value can make drifts in one direction or can cause destabilization
- Changing the kp value controls how strongly the car react to being tilted
- Stability of the car.
- Control and system parameters.

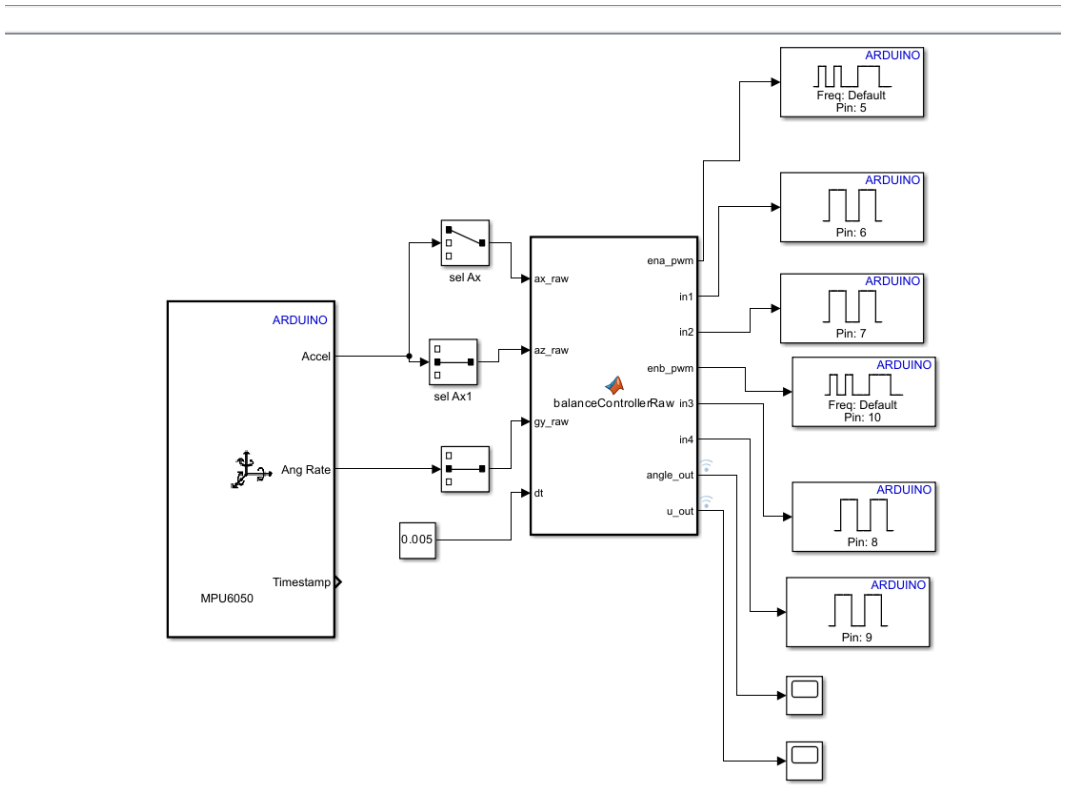
PID CONTROLLER DESIGN

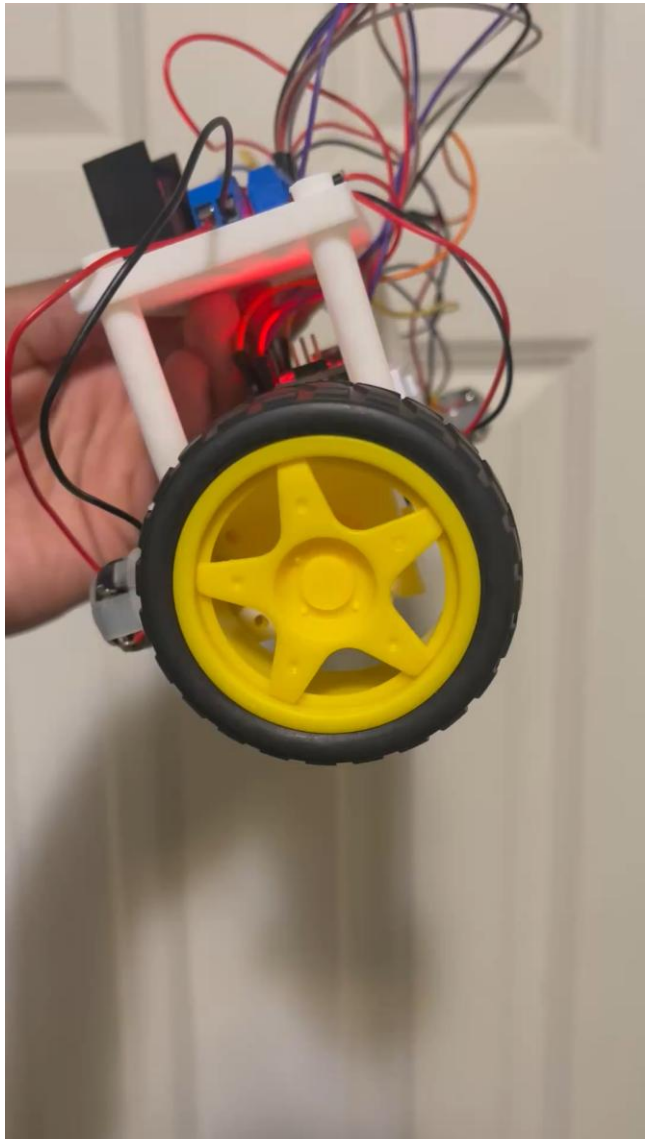


- Objective: Maintain car balance ($\theta \approx 0$) with minimal overshoot and short settling time.
- PID Structure:
  - P reduces error magnitude
  - I removes steady-state error
  - D improves damping and reduces oscillation
- Turning Approach
  - Start with P-only for stability
  - Add D to reduce oscillation
  - Add I to eliminate steady-state error
- Final
- We find that when our kp = 15, ki = 5 and kd = 15. Our car was almost about to be stable.

SIMSCAPE

- This is our block diagram
- Selector was used
  that acceleramotor can be connected
  to ax and az.
- Arduino interface and MATLAB code.
- We ran the code in the Arduino IDE.





RESULTS

- Demo of our self-balancing car in motion
- Plot of our angle is of set.
- We will show how the motor work.
- The plot showcased.

REFERENCES

[1] Minnesota State University, Mankato, “ME 341 Final Project Guidelines,” Department of Mechanical Engineering, 2025.

[2] Onshape, "Onshape for Startups," [Online]. Available: https://www.onshape.com/en/startups/apply

[3] *maclain-makanda/arduino-self-balancing-robot-master*. (Jan. 14, 2021). Makefile. Maclain Makanda. Accessed: Dec. 03, 2025. [Online]. Available: https://github.com/maclain-makanda/arduino-self-balancing-robot-master

DISCUSSION

To achieve long-term balance for the car, several improvements can be made:

- Upgrade to a more advanced microcontroller
- Higher quality motors
- Implement real-world testing
- Perform detailed calculations to measure parameters like inertia, mass, and other key factors for greater accuracy
- Use a more efficient gyroscope for better stability
- Add remote monitoring capabilities.

CONCLUSION

- Different values of Kp, Ki, and Kd directly affect the shape and behavior of the response plot
- The car was able to achieve short-term stability
- Maintaining long-term stability proved challenging due to real-world limitations, which require highly precise mechanical and electrical design

REFERENCES

[1] Minnesota State University, Mankato, “ME 341 Final Project Guidelines,” Department of Mechanical Engineering, 2025.

[2] Onshape, "Onshape for Startups," [Online]. Available: https://www.onshape.com/en/startups/apply

[3] *maclain-makanda/arduino-self-balancing-robot-master*. (Jan. 14, 2021). Makefile. Maclain Makanda. Accessed: Dec. 03, 2025. [Online]. Available: https://github.com/maclain-makanda/arduino-self-balancing-robot-master